

MERN CheatSheet





TABLE OF CONTENTS

1. Introduction to MERN Stack

- What is MERN Stack
- Components: MongoDB, Express.js, React, Node.js
- Why Use MERN
- MERN vs Other Stacks
- Real-World Use Cases

2. Node.js Basics

- What is Node.js
- Installing Node.js and npm
- Creating Your First Node App
- Built-in Modules
- Using External Packages (npm install)

3. Express.js Essentials

- What is Express.js
- Setting Up Express Server
- Routes (GET, POST, PUT, DELETE)
- Handling Requests & Responses
- Middleware
- Express Router
- Serving Static Files

4. MongoDB & Mongoose

- What is MongoDB
- Connecting MongoDB with Mongoose
- Creating Schemas and Models
- CRUD Operations
- Relationships in Mongoose
- Validations and Defaults
- MongoDB Atlas Setup

5. REST APIs with Express & MongoDB

- What is an API
- Creating RESTful Routes
- Using Postman for Testing
- Connecting Routes to MongoDB
- Error Handling in APIs
- Project Folder Structure



TABLE OF CONTENTS

6. React Basics

- What is React
- Setting Up React Project
- JSX Syntax
- Components
- Props and State
- useState and useEffect
- Conditional Rendering

7. React Advanced Concepts

- Handling Forms and Inputs
- Lifting State Up
- useContext
- useRef and DOM Access
- useReducer
- Custom Hooks
- React Router (v6)

8. Connecting React to Express API

- Fetching Data from API
- Displaying API Data
- Submitting Form Data
- Handling Loading and Errors
- CORS and Proxy Setup

9. Authentication and Authorization

- Auth vs Authorization
- Register & Login APIs
- Password Hashing with bcrypt
- JWT Token
- Protecting Express Routes
- Storing Tokens in Frontend
- Protected Routes in React

10. Project Deployment

- Preparing Backend for Deployment
- Building React Frontend
- Connecting Frontend & Backend
- Deployment with Render / Vercel / Railway
- Environment Variables
- Domain & Hosting



TABLE OF CONTENTS

11. Bonus Tips

- Common Beginner Mistakes
- Folder Structure Best Practices
- Using Postman Collections
- Debugging Tips
- Resources to Learn More

TAP TO GET LINK



M



E



R



N



1. INTRODUCTION TO MERN STACK

1.1 What is MERN Stack?

- MERN Stack is a set of four powerful technologies used to build full-stack web applications.
 - The word "MERN" is made from the first letters of each tool:
 - **M**: MongoDB
 - **E**: Express.js
 - **R**: React
 - **N**: Node.js
-
- All four work together to help developers build apps that have a front-end (what the user sees) and a back-end (how data is stored and managed).
 - MERN is full JavaScript-based, so if you know JavaScript, you can learn the entire stack easily.

1.2 Components of MERN (MongoDB, Express.js, React, Node.js)

- Let's understand each part of MERN Stack one by one:

MongoDB (Database)

- It stores data in a format similar to JSON (key-value pairs).
- It's a NoSQL database, which means it's flexible and easy to use.
- Example: You can store user details, product data, messages, etc.

Express.js (Backend Web Framework)

- It runs on top of Node.js.
- It handles server-side logic like routes, requests, and responses.
- Helps you create APIs (paths that send/receive data).
- Example: When someone logs in, Express handles the login request.

React (Frontend Library)

- Created by Facebook.
- Helps you build the user interface (UI) — the buttons, forms, pages users see.
- It uses something called components to make UI reusable.
- Example: A login form or product list page.

Node.js (Runtime Environment)

- Lets JavaScript run outside the browser — on a server.
- It allows you to build the back-end part using JavaScript.
- It's very fast and handles many tasks at once.



1. INTRODUCTION TO MERN STACK

1.3 Why Use the MERN Stack?

- Here are simple reasons why developers like the MERN Stack:
 1. Single Language – You only need to know JavaScript for front-end and back-end.
 2. Free and Open Source – All tools are free to use and have big communities.
 3. Fast Development – React and Node.js are fast and make apps responsive.
 4. Real-Time Updates – You can easily create apps that update without refreshing (like chat apps).
 5. Strong Community Support – Thousands of tutorials, tools, and answers are available.

1.4 MERN vs Other Stacks (MEAN, LAMP, etc.)

- Here's a simple comparison:

Stack	Front-End	Back-End	Database	Language
MERN	React	Node.js + Express.js	MongoDB	JavaScript
MEAN	Angular	Node.js + Express.js	MongoDB	JavaScript
LAMP	HTML/CSS/JS	PHP	MySQL	Mixed (PHP, SQL, JS)

- **MERN** uses React (easy, fast)
- **MEAN** uses Angular (more structured)
- **LAMP** is old and uses PHP & MySQL

Why MERN wins?

- It's modern, flexible, JavaScript-based, and great for building dynamic web apps like social media, eCommerce, and dashboards.

1.5 Real-World Use Cases

- Here are some examples where **MERN Stack** is used:
 1. **Social Media Apps**: Like Facebook or Instagram – posts, likes, comments.
 2. **eCommerce Websites**: Like Amazon – users, products, payments, carts.
 3. **Blog Platforms**: Like Medium – write, edit, comment on articles.
 4. **Project Management Tools**: Like Trello – tasks, users, drag-drop features.
 5. **Real-Time Chat Apps**: Like WhatsApp – messages update instantly.



2. NODE.JS BASICS

2.1 What is Node.js?

- Node.js is a JavaScript runtime – this means it lets you run JavaScript code outside the browser, like on your computer or a server.
- Before Node.js, JavaScript could only run in browsers (like Chrome). But with Node.js,

you can use JavaScript to:

- Create servers
 - Handle files
 - Connect to databases
 - Build full web applications
-
- Node.js is built on Chrome's V8 engine, which makes it very fast and non-blocking (it doesn't wait for one task to finish before starting the next).

In simple words:

- Node.js lets you use JavaScript to build the backend of your website.

2.2 Installing Node.js and npm

- To write Node.js code on your computer, you first need to install it.

Step-by-Step:

1. Go to <https://nodejs.org>
2. Download the LTS (Recommended) version
3. Install it like any other software

After installing, check if it worked:

- Open Command Prompt or Terminal
- Type:

```
bash
node -v
```

- This shows the Node.js version
- Then type:

```
bash
npm -v
```

- This shows the npm version

What is npm?

- npm = Node Package Manager
- It helps you install and manage packages (extra tools/code written by others)

2.3 Creating Your First Node.js App

- Let's build a simple Node.js app:

Step 1: Create a file

- Make a file called app.js



2. NODE.JS BASICS

2.3 Creating Your First Node.js App

Step 2: Add this code

```
javascript  
console.log("Hello, Node.js!");
```

Step 3: Run it

- Open terminal, go to the folder, and run:

```
bash  
node app.js
```

- You will see:

```
Hello, Node.js!
```

- That means Node.js is working!

2.4 Common Built-in Modules

- Node.js has some built-in tools (called modules). You don't need to install them. Just use them with `require()`.

Some useful ones:

1. fs (File System) – For reading/writing files

```
javascript  
const fs = require('fs');  
fs.writeFileSync("hello.txt", "This is Node.js");
```

2. http – For making a simple server

```
javascript  
const http = require('http');  
const server = http.createServer((req, res) => {  
  res.end("Hello from server");  
});  
server.listen(3000);
```

3. path – To work with file paths

```
javascript  
const path = require('path');  
console.log(path.basename('/folder/file.txt')); // file.txt
```



2. NODE.JS BASICS

2.5 Using External Packages (npm install)

- You can add extra features to your app by using packages created by others.

Example: Install a package called chalk (adds colors to console output)

1. First, run this in terminal:

```
bash
npm init -y
```

- It creates a package.json file (tracks your project and packages)

2. Install chalk:

```
bash
npm install chalk
```

3. Use it in your code:

```
javascript
const chalk = require('chalk');
console.log(chalk.green('Success!'));
```

- You can find thousands of free packages at <https://www.npmjs.com>



3. EXPRESS.JS ESSENTIALS

3.1 What is Express.js?

- Express.js is a web framework built on top of Node.js.
- It helps you create web servers easily and fast.
- Without Express, you have to write a lot of code to handle routes and requests.
- With Express, it becomes super simple.
- Think of Express as a toolbox that gives you shortcuts to build web apps.

3.2 Setting Up Express Server

Step-by-Step:

1. Create a new folder for your project
2. Open terminal in that folder
3. Run:

```
bash  
npm init -y
```

4. Install Express:

```
bash  
npm install express
```

5. Create a file app.js and add this code:

```
javascript  
  
const express = require('express');  
const app = express();  
  
app.get('/', (req, res) => {  
  res.send("Hello from Express!");  
});  
  
app.listen(3000, () => {  
  console.log("Server running on http://localhost:3000");  
});
```

6. Run the app:

```
bash  
node app.js
```

- Open browser and go to <http://localhost:3000>



3. EXPRESS.JS ESSENTIALS

3.3 Routes (GET, POST, PUT, DELETE)

- Routes are paths like /home, /about, etc.
- Each route can respond to different HTTP methods:

Common ones:

- GET – Get data
- POST – Send new data
- PUT – Update data
- DELETE – Remove data

Example:

```
javascript

app.get('/hello', (req, res) => {
  res.send("Hello World");
});

app.post('/submit', (req, res) => {
  res.send("Data received");
});

app.put('/edit', (req, res) => {
  res.send("Data updated");
});

app.delete('/delete', (req, res) => {
  res.send("Data deleted");
});
```

3.4 Handling Requests & Responses

- req = request object (what the user sends)
- res = response object (what we send back)

Example:

```
javascript

app.get('/user', (req, res) => {
  res.send("User info");
});
```

- You can also access query and body from the request:

```
javascript

app.get('/search', (req, res) => {
  const query = req.query.name;
  res.send("You searched for " + query);
});
```



3. EXPRESS.JS ESSENTIALS

3.5 Using Middleware

- Middleware is a function that runs before the request is processed.

It can:

- Log data
- Check authentication
- Parse JSON

Example:

```
javascript

app.use(express.json()); // Middleware to parse JSON data

app.post('/data', (req, res) => {
  console.log(req.body);
  res.send("Data received");
});
```

- You can also create your own middleware:

```
javascript

const logger = (req, res, next) => {
  console.log(req.method, req.url);
  next(); // move to next function
};

app.use(logger);
```

3.6 Express Router

- Routers help split your code into smaller files.
- Useful for big apps with many routes.

Example:

1. Create a file routes/user.js

```
javascript

const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send("User Home");
});

router.get('/profile', (req, res) => {
  res.send("User Profile");
});

module.exports = router;
```




3. EXPRESS.JS ESSENTIALS

3.6 Express Router

Example:

2. In app.js, use the router:

```
javascript

const userRoutes = require('./routes/user');
app.use('/user', userRoutes);
```

- Now, /user/ and /user/profile will work.

3.7 Serving Static Files

- You can serve HTML, CSS, images using Express.

Step-by-Step:

1. Create a folder public
2. Add files like index.html, style.css, etc.
3. Add this in app.js:

```
javascript

app.use(express.static('public'));
```

- Now if you go to <http://localhost:3000/index.html>, the file will open.



4. MONGODB & MONGOOSE

4.1 What is MongoDB?

- MongoDB is a NoSQL database used to store data in a flexible, easy-to-read format called documents.
- These documents are stored inside collections, similar to tables in SQL.
- Each document is written in JSON-like format, using key-value pairs:

```
javascript

const userRoutes = require('./routes/user');
app.use('/user', userRoutes);
```

MongoDB is:

- Schema-less (you don't need to define the structure of data in advance)
- Scalable (can handle large amounts of data)
- Flexible (you can add/remove fields anytime)

4.2 Connecting MongoDB with Mongoose

- Mongoose is a Node.js library that makes working with MongoDB easier.

It helps you:

- Define schemas (structure for your data)
- Perform validations
- Run queries easily

Step-by-Step:

1. First, install Mongoose:

```
bash

npm install mongoose
```

2. Then, connect to MongoDB:

```
javascript

const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error(err));
```

- You can also use MongoDB Atlas for online databases (covered in 4.7).

4.3 Creating Schemas and Models

- A Schema is like a blueprint for your data (what fields it should have).
- A Model is like a tool to interact with the collection.

Example:

```
javascript

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  isStudent: Boolean
});

const User = mongoose.model('User', userSchema);
```



4. MONGODB & MONGOOSE

- Now User can be used to create, read, update, delete users in MongoDB.

4.4 CRUD Operations

- CRUD = Create, Read, Update, Delete

Create:

```
javascript

const user = new User({ name: "John", age: 25, isStudent: true });
user.save();
```

Read:

```
javascript

User.find().then(users => console.log(users));
User.findOne({ name: "John" });
```

Update:

```
javascript

User.updateOne({ name: "John" }, { age: 30 });
```

Delete:

```
javascript

User.deleteOne({ name: "John" });
```

4.5 Relationships in Mongoose

- MongoDB is NoSQL, so it does not use "joins" like SQL, but we can create relationships using references.

Example:

- Post Schema

```
javascript

const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }
});
```

- Then, when getting posts, you can populate user data:

```
javascript

Post.find().populate('author').then(posts => console.log(posts));
```



4. MONGODB & MONGOOSE

4.6 Validations and Defaults

- Schemas can have validations and default values.

Example:

```
javascript

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: { type: Number, min: 1, max: 100 },
  createdAt: { type: Date, default: Date.now }
});
```

- required: true – field must be present
- min and max – for number range
- default – sets default value if nothing is given

4.7 MongoDB Atlas Setup

- MongoDB Atlas is a cloud version of MongoDB – you can access it from anywhere.

Steps to use MongoDB Atlas:

1. Go to: <https://www.mongodb.com/cloud/atlas>
2. Create a free account
3. Create a new project and a free cluster
4. Add a database user (username and password)
5. Whitelist your IP (0.0.0.0 for access from anywhere)
6. Copy the connection string:

```
perl

mongodb+srv://<username>:<password>@cluster0.xxxxxx.mongodb.net/mydatabase
```

7. Use it in your Mongoose code:

```
javascript

mongoose.connect('your-connection-string')
  .then(() => console.log('Connected to MongoDB Atlas'))
  .catch(err => console.error(err));
```



5. REST APIS WITH EXPRESS & MONGODB

5.1 What is an API?

- API stands for Application Programming Interface.
- It allows two systems (like frontend and backend) to talk to each other.
- A REST API is a type of API that follows certain rules to handle data using standard HTTP methods:

Method	Purpose	Example
GET	Read data	Get user list
POST	Create data	Add a new product
PUT	Update data	Update user info
DELETE	Delete data	Remove a product

Example:

- A mobile app sends a POST request to /users to create a new user.

5.2 Creating RESTful Routes

- Using Express.js, you can create RESTful API routes easily.
- Example for a User model:

```
javascript

const express = require('express');
const router = express.Router();
const User = require('../models/User');

// Create - POST
router.post('/', async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.send(user);
});

// Read - GET
router.get('/', async (req, res) => {
  const users = await User.find();
  res.send(users);
});

// Update - PUT
router.put('/:id', async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.send(user);
});

// Delete - DELETE
router.delete('/:id', async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.send({ message: 'User deleted' });
});

module.exports = router;
```



5. REST APIS WITH EXPRESS & MONGODB

- Then in app.js:

```
javascript

const userRoutes = require('./routes/user');
app.use('/users', userRoutes);
```

Now your API has:

- POST /users
- GET /users
- PUT /users/:id
- DELETE /users/:id

5.3 Using Postman for Testing

- Postman is a free tool to test APIs.

Steps to use Postman:

1. Download it from <https://www.postman.com>
2. Open Postman and create a new request
3. Select method (GET, POST, etc.)
4. Enter URL like: `http://localhost:3000/users`
5. If POST or PUT, go to Body > raw > JSON, and add data:

```
json

{
  "name": "John",
  "age": 25
}
```

6. Click Send and see the response.

5.4 Connecting Routes to MongoDB

You need:

- MongoDB (or Atlas)
- Mongoose (installed via `npm install mongoose`)
- A Model (like User)

Example Model:

```
javascript

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});

module.exports = mongoose.model('User', userSchema);
```

- Your route files (like shown in 5.2) will use this model to store and fetch data.



5. REST APIS WITH EXPRESS & MONGODB

5.5 Error Handling in APIs

- Always handle errors to avoid crashing your server.

Example:

```
javascript

router.post('/', async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).send(user);
  } catch (err) {
    res.status(400).send({ error: err.message });
  }
});
```

You can use:

- try...catch blocks
- res.status(code).send(message)
- Custom error messages

5.6 Project Folder Structure

- Here's a simple structure for an Express + MongoDB REST API:

```
pgsql

myapp/
├── models/
│   └── User.js          # Mongoose schema
├── routes/
│   └── user.js          # All user-related routes
├── app.js               # Main entry file
└── package.json         # Project info and dependencies
```

Bonus Tip:

- You can add a config folder to store the MongoDB URL and environment variables.



6. REACT BASICS

6.1 What is React?

- React is a JavaScript library used to build user interfaces (UI).
- It was created by Facebook and is now one of the most popular tools to build web apps.

React is:

- Component-based – break UIs into small reusable pieces
- Fast and flexible – only updates the parts of the page that change
- Used in many real-world apps like Instagram, Facebook, Netflix

React works with something called a Virtual DOM – a fast version of the webpage stored in memory that makes updates faster.

6.2 Setting Up React with Vite or Create React App

You can create a new React app using 2 popular tools:

- Create React App (CRA) – traditional setup
- Vite – faster, modern setup

Option 1: Using Create React App

```
bash
npx create-react-app my-app
cd my-app
npm start
```

Option 2: Using Vite (recommended for speed)

```
bash
npm create vite@latest my-app --template react
cd my-app
npm install
npm run dev
```

- Then open browser at <http://localhost:5173> (for Vite) or <http://localhost:3000> (for CRA).

6.3 JSX Syntax

- JSX = JavaScript + XML
- In React, we write HTML inside JavaScript using JSX.

Example:

```
jsx
function App() {
  return <h1>Hello, React!</h1>;
}
```

Important JSX rules:

- Use camelCase for HTML attributes (className instead of class)
- Return only one parent element
- Use {} to insert JS inside JSX



6. REACT BASICS

6.3 JSX Syntax

- Example with JS inside JSX:

```
jsx
const name = "Rudra";
return <h2>Hello, {name}</h2>;
```

6.4 Components (Functional and Class)

- Component = small building block of UI
- There are two types of components:

1. Functional Component (Most Used)

```
jsx
function Welcome() {
  return <h1>Welcome!</h1>;
}
```

2. Class Component (Older Way)

```
jsx
import React, { Component } from 'react';

class Welcome extends Component {
  render() {
    return <h1>Welcome!</h1>;
  }
}
```

6.5 Props and State

Props (Short for Properties)

- Used to pass data from parent to child component
- Props are read-only

```
jsx
function Greeting(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

- Call it like:

```
jsx
<Greeting name="Rudra" />
```

State

- Used to store and manage changing data inside a component
- Like a variable that React watches

```
jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;
}
```



6. REACT BASICS

6.6 useState and useEffect Hooks

useState

- Adds state to a functional component

```
jsx
const [value, setValue] = useState(0);
```

useEffect

- Runs code when a component loads, updates, or unmounts
- Often used for API calls, timers, etc.

```
jsx
import { useEffect } from 'react';

useEffect(() => {
  console.log("Component loaded");
}, []);
```

- “[]” means it runs only once when the component mounts.

6.7 Conditional Rendering

- React can show or hide elements based on conditions.

Example using if:

```
jsx
if (loggedIn) {
  return <h1>Welcome back!</h1>;
} else {
  return <h1>Please log in</h1>;
}
```

Using ternary:

```
jsx
return <h1>{loggedIn ? "Welcome" : "Login first"}</h1>;
```

Using && (show only if true):

```
jsx
{show && <p>This is visible</p>}
```



7. REACT ADVANCED CONCEPTS

7.1 Handling Forms and Inputs

- React forms help collect user input like names, emails, or passwords.

Example of a simple form:

```
import { useState } from 'react';

function SimpleForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Submitted: " + name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter name"
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Key Concepts:

- value: tracks the input value
- onChange: updates the value as user types
- onSubmit: runs when the form is submitted

7.2 Lifting State Up

- When two components need to share the same data, move the state to their common parent.

Example:

Parent component

```
import { useState } from 'react';
import InputBox from './InputBox';
import Display from './Display';

function Parent() {
  const [text, setText] = useState("");

  return (
    <>
      <InputBox text={text} setText={setText} />
      <Display text={text} />
    </>
  );
}
```



7. REACT ADVANCED CONCEPTS

- *Child components*

```
function InputBox({ text, setText }) {  
  return <input value={text} onChange={(e) => setText(e.target.value)} />;  
}  
  
function Display({ text }) {  
  return <p>You typed: {text}</p>;  
}
```

- This is called "lifting state up" because state lives in the parent and is passed down.

7.3 useContext for Global State

- useContext helps avoid prop drilling (passing props through many layers).

Step-by-step:

1. Create a context file:

```
import { createContext } from 'react';  
  
export const AppContext = createContext();
```

2. Wrap your app with the provider:

```
<AppContext.Provider value={{ user: "Rudra" }}>  
  <MyComponent />  
</AppContext.Provider>
```

3. Use the context in a child:

```
import { useContext } from 'react';  
import { AppContext } from './AppContext';  
  
function MyComponent() {  
  const { user } = useContext(AppContext);  
  return <p>Hello, {user}</p>;  
}
```

7.4 useRef and DOM Access

useRef is used to:

- Access DOM elements directly
- Store values that don't trigger re-renders

Example 1: Accessing input field



7. REACT ADVANCED CONCEPTS

7.4 useRef and DOM Access

Example 1: Accessing input field

```
import { useRef } from 'react';

function InputFocus() {
  const inputRef = useRef();

  const handleClick = () => {
    inputRef.current.focus();
  };

  return (
    <>
      <input ref={inputRef} />
      <button onClick={handleClick}>Focus</button>
    </>
  );
}
```

Example 2: Storing a value

```
const count = useRef(0);
```

7.5 useReducer

- useReducer is like useState, but better for complex state logic.

Example: Counter with reducer

```
import { useReducer } from 'react';

const reducer = (state, action) => {
  switch (action.type) {
    case 'increment': return state + 1;
    case 'decrement': return state - 1;
    default: return state;
  }
};

function Counter() {
  const [count, dispatch] = useReducer(reducer, 0);

  return (
    <>
      <p>{count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </>
  );
}
```



7. REACT ADVANCED CONCEPTS

7.6 Custom Hooks

- Custom hooks let you reuse logic across components.

Example: useTitle hook

```
import { useEffect } from 'react';

function useTitle(title) {
  useEffect(() => {
    document.title = title;
  }, [title]);
}
```

- Use it in a component:

```
function About() {
  useTitle("About Page");
  return <h1>About</h1>;
}
```

7.7 React Router (v6) – Navigation & Routing

- React Router helps you build single-page apps (SPAs) with multiple views.

Step-by-Step Setup:

1. Install:

```
npm install react-router-dom
```

2. Basic Setup:

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}
```



8. CONNECTING REACT TO EXPRESS API

8.1 Fetching Data with fetch() or Axios

- React can connect to a backend (like Express) using two popular methods:

1. Using fetch() (built-in)

```
jsx
useEffect(() => {
  fetch('http://localhost:3000/users')
    .then((res) => res.json())
    .then((data) => setUsers(data));
}, []);
```

2. Using Axios (more powerful)

1. First install Axios:

```
bash
npm install axios
```

2. Then use it:

```
jsx
import axios from 'axios';

useEffect(() => {
  axios.get('http://localhost:3000/users')
    .then((res) => setUsers(res.data))
    .catch((err) => console.error(err));
}, []);
```

8.2 Displaying API Data in Components

- Once data is fetched, store it in state and use .map() to show it.

Example:

```
jsx
const [users, setUsers] = useState([]);

return (
  <div>
    <h2>User List:</h2>
    <ul>
      {users.map((user) => (
        <li key={user._id}>{user.name}</li>
      ))}
    </ul>
  </div>
);
```

- Make sure each item has a key (like _id or index).



8. CONNECTING REACT TO EXPRESS API

8.3 Submitting Form Data to Backend

- To send data to your Express API, use POST.

Example using fetch:

```
jsx

const handleSubmit = async () => {
  await fetch('http://localhost:3000/users', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name, email }),
  });
};
```

- Example using Axios:

```
jsx

axios.post('http://localhost:3000/users', { name, email })
  .then((res) => console.log(res.data))
  .catch((err) => console.error(err));
```

8.4 Handling Loading and Errors

- To show loading spinners or error messages:

Example:

```
jsx

const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState("");

useEffect(() => {
  fetch('http://localhost:3000/users')
    .then((res) => res.json())
    .then((data) => {
      setUsers(data);
      setLoading(false);
    })
    .catch((err) => {
      setError("Something went wrong");
      setLoading(false);
    });
}, []);
```

- Render conditionally:

```
jsx

{loading && <p>Loading...</p>}
{error && <p>{error}</p>}
{!loading && users.map(user => <div>{user.name}</div>)}
```

8.5 CORS and Proxy Setup

- By default, browsers block cross-origin requests.
- If your frontend is on localhost:5173 and backend is on localhost:3000, you may see a CORS error.



8. CONNECTING REACT TO EXPRESS API

Option 1: Fix in Express (recommended)

- Install CORS in your Express backend:

```
bash
npm install cors
```

- Add this in your app.js:

```
javascript
const cors = require('cors');
app.use(cors());
```

Option 2: Use Proxy in React (only in development)

- In your React project's package.json, add:

```
json
"proxy": "http://localhost:3000"
```

- Now instead of:

```
javascript
fetch('http://localhost:3000/users')
```

- You can just use:

```
javascript
fetch('/users')
```



9. AUTHENTICATION AND AUTHORIZATION

9.1 What is Authentication vs Authorization?

- Let's break it down simply:

Term	Meaning
Authentication	Are you the person you say you are? (Login/Register)
Authorization	Do you have permission to do this? (Admin-only actions)

Example:

- You log in to a website = ☒ Authentication
- You try to open the admin dashboard = ☒ Authorization check

9.2 Register & Login APIs

In Express, you can create two routes:

- /register – to create a new user
- /login – to verify and allow access

Register Route (POST /register):

```
javascript

const bcrypt = require('bcrypt');
const User = require('../models/User');

app.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);

  const newUser = new User({ name, email, password: hashedPassword });
  await newUser.save();

  res.status(201).send("User registered");
});
```

- Login Route (POST /login):

```
javascript

const jwt = require('jsonwebtoken');

app.post('/login', async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user) return res.status(400).send("User not found");

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).send("Invalid credentials");

  const token = jwt.sign({ id: user._id }, 'secretKey');
  res.send({ token });
});
```



9. AUTHENTICATION AND AUTHORIZATION

9.3 Password Hashing with bcrypt

- Hashing = converting the password into unreadable form

Steps:

- Install bcrypt:

```
bash
npm install bcrypt
```

- Use `bcrypt.hash(password, saltRounds)` to encrypt
- Use `bcrypt.compare(plain, hashed)` to check passwords

This keeps user passwords safe in the database.

9.4 JWT Token Generation and Verification

- JWT = JSON Web Token
- It's like a digital ID card. Once a user logs in, you give them a token.
- To generate a token:

```
javascript
const token = jwt.sign({ id: user._id }, 'secretKey', { expiresIn: '1h' });
```

- To verify the token later:

```
javascript
const decoded = jwt.verify(token, 'secretKey');
```

- JWT is sent with every protected request.

9.5 Protecting Routes in Express

- To make a route private, use middleware to check the JWT token:

```
javascript
function verifyToken(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(403).send("Token missing");

  const token = authHeader.split(' ')[1]; // Bearer <token>
  try {
    const decoded = jwt.verify(token, 'secretKey');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).send("Invalid Token");
  }
}
```

- Use it on any route:

```
javascript
app.get('/profile', verifyToken, (req, res) => {
  res.send("Private profile accessed");
});
```



9. AUTHENTICATION AND AUTHORIZATION

9.6 Storing Tokens in Frontend (LocalStorage or Cookies)

- After login, the frontend gets a token. Where do we keep it?

Option 1: LocalStorage

```
javascript

localStorage.setItem("token", response.data.token);
```

- Easy to use
- But vulnerable to XSS attacks (not good for sensitive apps)

Option 2: Cookies (with httpOnly)

- Safer for production
- Requires backend to send cookies using res.cookie()

9.7 Protecting Routes in React

- Once user logs in, you can show/hide components or pages based on the token.

Simple example:

```
javascript

const token = localStorage.getItem("token");

if (!token) {
  return <Navigate to="/login" />;
}
```

- ProtectedRoute Component:

```
jsx

import { Navigate } from 'react-router-dom';

function ProtectedRoute({ children }) {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" />;
}
```

- Use it in routes:

```
jsx

<Route path="/dashboard" element={
  <ProtectedRoute>
    <Dashboard />
  </ProtectedRoute>
} />
```



10. PROJECT DEPLOYMENT (FULL MERN APP)

10.1 Preparing Backend for Deployment

- Before deploying the backend (Express + MongoDB):

Steps:

1. Remove hardcoded URLs – use environment variables
2. Use production-ready database (like MongoDB Atlas)
3. Enable CORS properly

```
js
const cors = require('cors');
app.use(cors({ origin: 'https://your-frontend-domain.com' }));
```

4. Update package.json

- Add this script:

```
json
"start": "node index.js"
```

5. Add .env file:

```
ini
PORT=5000
MONGODB_URI=your-atlas-url
JWT_SECRET=your-secret-key
```

10.2 Preparing React Frontend for Build

- To deploy your React app:

Steps:

1. Open terminal in your React project folder
2. Run:

```
bash
npm run build
```

3. A new folder called build/ will be created. It contains the final static files.

10.3 Connecting Frontend and Backend

- You have two options:

Option 1: Same server

- Serve React build from Express backend

```
js
app.use(express.static('client/build'));

app.get('*', (req, res) => {
  res.sendFile(path.resolve(__dirname, 'client', 'build', 'index.html'));
});
```

Option 2: Deploy separately

- Frontend on Vercel/Netlify
- Backend on Render/Railway
- Connect using full URLs (https://your-api-domain.com/api/users)
- Make sure your backend supports CORS.



10. PROJECT DEPLOYMENT (FULL MERN APP)

10.4 Deploying with Render / Vercel / Railway

Deploy Backend on Render

1. Go to <https://render.com>
2. Connect your GitHub repo
3. Choose Web Service
4. Set build and start commands:

```
Build Command: npm install  
Start Command: npm start
```

5. Add environment variables under "Environment"

Deploy Frontend on Vercel

1. Go to <https://vercel.com>
 2. Connect GitHub repo
 3. Select root folder of frontend (if in /client)
 4. Vercel will auto-detect React and deploy it
- Railway is similar to Render but more developer-friendly
 - <https://railway.app>

10.5 Environment Variables in Production

- For Express:
- Use .env file and dotenv package:

```
require('dotenv').config();  
const PORT = process.env.PORT;
```

- For React:
- Use REACT_APP_ prefix:

```
REACT_APP_API_URL=https://your-api.onrender.com
```

- Access it in React:

```
process.env.REACT_APP_API_URL
```

- You must rebuild React if you change .env.



10. PROJECT DEPLOYMENT (FULL MERN APP)

10.6 Domain & Hosting Suggestions

- Free Hosting Options:

Service	Best For	Free Plan
Vercel	Frontend	✓ Yes
Netlify	Frontend	✓ Yes
Render	Fullstack	✓ Yes
Railway	Backend/API	✓ Yes
MongoDB Atlas	Cloud Database	✓ Yes

Custom Domain (Optional)

- Buy from: GoDaddy, Namecheap
- Connect domain to Vercel/Render in settings
- Use DNS records (A/CNAME) to point your domain



11. BONUS TIPS (FOR MERN STACK BEGINNERS)

11.1 Common Beginner Mistakes in MERN

Mistake	Why It's a Problem
✗ Hardcoding URLs	Use environment variables instead of writing full URLs in code.
✗ Forgetting await	Without await, your code runs before data is ready.
✗ Using wrong HTTP methods	Use GET for reading, POST for creating, etc.
✗ Not handling errors	If APIs fail, the user gets no feedback.
✗ Skipping CORS setup	Causes "CORS" errors when frontend can't access backend.
✗ Confusing find() vs findOne()	find() returns an array, findOne() returns one object.

11.2 Folder Structure Best Practices

- A clean structure helps you find files quickly and work in teams easily.
- Recommended MERN Folder Layout:

```
bash

project-root/
├── client/                # React frontend
│   ├── public/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── services/      # API calls
│       ├── context/       # Global state
│       └── App.js
├── server/               # Express backend
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── middleware/
│   └── server.js
├── .env                  # Environment variables
├── package.json
└── README.md
```

11.3 Using Postman Collections

- Postman is not just for testing; you can save and organize requests in Collections.

Why use Postman Collections?

- Group all endpoints (like User APIs, Product APIs)
- Save headers, tokens, and request bodies
- Share collection with team or students

Steps:

1. In Postman, create a Collection
2. Add each API request inside it
3. Export the collection (.json file) to reuse/share



11. BONUS TIPS (FOR MERN STACK BEGINNERS)

11.4 Debugging Tips

- Every developer faces bugs — here's how to fix them easily:

Frontend (React) Debugging:

- Use `console.log()` to check values
- Use browser DevTools > Network tab to check API errors
- Check `.env` variables and restart after changes

Backend (Express) Debugging:

- Use `console.log()` to inspect request data
- Add error-handling middleware
- Use tools like Postman to isolate backend problems

General Tip:

- If something doesn't work: check file paths, ports, and CORS. 80% of errors are here.

11.5 Resources to Learn More

Documentation:

- React Docs: <https://reactjs.org>
- Express Docs: <https://expressjs.com>
- MongoDB Docs: <https://www.mongodb.com/docs>
- Mongoose Docs: <https://mongoosejs.com/docs>

YouTube Channels:

- Code with Harry (Hindi + English mix)
- Traversy Media
- The Net Ninja
- freeCodeCamp
- Anuj Bhaiya (Hindi MERN content)

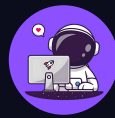
Blogs and Courses:

- [GeeksforGeeks.org](https://www.geeksforgeeks.org)
- [JavaScript.info](https://javascript.info)
- Udemy MERN Stack Courses (Search for “MERN Stack Bootcamp”)
- [MDN Web Docs](https://developer.mozilla.org)



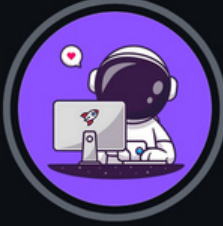
MERN STACK SUMMARY (MODULES 1-11)

Module	Topic	Summary
1	Intro to MERN	What MERN is, its 4 parts (MongoDB, Express, React, Node), and why it's popular.
2	Node.js Basics	Run JavaScript on the server, install packages, and build simple backend apps.
3	Express.js	Create server, handle routes (GET/POST), middleware, and serve static files.
4	MongoDB & Mongoose	Store data, connect with Mongoose, create schemas, and do CRUD operations.
5	REST APIs	Build APIs using Express + MongoDB, test with Postman, and structure folders.
6	React Basics	Build UI using components, JSX, state, props, and lifecycle with hooks.
7	React Advanced	Use Context, useRef, useReducer, custom hooks, and routing with React Router.
8	React + Express	Connect frontend and backend using fetch/Axios, handle CORS, and submit forms.
9	Auth & JWT	Register/login users, hash passwords, generate tokens, and protect routes.
10	Deployment	Host frontend/backend with Vercel/Render, use environment variables and domains.
11	Bonus Tips	Avoid mistakes, use clean folder structure, Postman, debugging, and learning resources.



Was this post helpful ?

 **coders_section** 



Coding | Programming | HTML | CSS
168 posts **31K** followers **28** following



Follow Our 2nd Account

 **webstrom_tech**  



Coding | Programming | HTML | CSS
3 posts **80** followers **1** following



Follow For More



Tap Here